PulmoScan AI

VIVA AND INTERVIEW PREPARATION

Viva Questions and Answers

PulmoScan AI: Intelligent Pneumonia Detection Using Convolutional Neural Networks

PREPARED FOR	Tanisha Dey
PROGRAMME	B.Tech, Data Science
DATE	November 2026
QUESTIONS	55 questions in 7 topics

Educational prototype. Not a medical device and not clinically validated.

How to use this document

This document contains 55 questions an examiner or interviewer is likely to ask about PulmoScan AI, each with a short answer written to be said aloud. Where it helps, the answer names the file to open if you are asked to show the code.

- Read the answers, then practise giving them in your own words. An examiner can tell a memorised sentence from an understood one.
- Know the numbers on the next page by heart. They are the measured results of the committed model.
- Be direct about the weak result. The test accuracy is lower than the validation accuracy, and you can explain exactly why. An honest, well-reasoned answer is stronger than a high number you cannot defend.
- If you retrain the model, the numbers change. Update them from the Model Insights page of the application.

Numbers to remember

Item	Value
Dataset	Kermany et al. 2018: 5,856 paediatric chest X-rays
Train / validation / test	4,484 / 748 / 624 images
Input size	160 x 160 pixels, grayscale
Architecture	4 convolutional blocks (16, 32, 64, 128 filters), global average pooling, dense 64, sigmoid
Parameters	106,193 (model file 1.37 MB)
Training	Adam, learning rate 0.001, batch 32, stopped at epoch 13, best epoch 8
Validation accuracy	95.2%
Test accuracy	**78.8%**
Test ROC-AUC	**0.930**
Pneumonia recall (sensitivity)	98.5% (384 of 390 caught)
Normal recall (specificity)	46.2% (126 of 234 wrongly flagged)
Confusion matrix	TN 108, FP 126, FN 6, TP 384
Deployment	Frontend on Vercel, backend (Docker) on Render, code on GitHub

> PulmoScan AI is an educational prototype. If asked whether it can be used on patients, the answer is no: it is not clinically validated.

Contents

How to use this document 2

Contents 3

A. Project overview 4

B. Data and preprocessing 4

C. The CNN 6

D. Evaluation 7

E. Grad-CAM 9

F. Backend, frontend and deployment 10

G. Limitations, ethics and future work 11

Quick formula sheet 12

A. Project overview

1 Explain your project in 30 seconds.

PulmoScan AI is a web application that classifies a chest X-ray as *Normal* or *Pneumonia* using a convolutional neural network I trained in TensorFlow/Keras. A FastAPI backend serves the model and returns the prediction, the probability of each class, and a Grad-CAM heatmap showing which regions influenced the decision. A React frontend, deployed as a static site on Vercel, provides the upload dashboard and a page with the measured evaluation metrics. It is an educational prototype, not a diagnostic tool.

2 What is the architecture of the whole system?

Three parts: (1) `ml/` - offline scripts that preprocess data, train and evaluate the model; (2) `backend/` - a FastAPI service that loads the saved model once and exposes `/api/predict`, `/api/model/insights`, `/api/health`; (3) `frontend/` - a React single-page app that calls those endpoints. The frontend is static files on Vercel (a Netlify configuration is also in the repository); the backend runs in a Docker container on Render.

3 Why can't the backend run on the same host as the frontend (Vercel/Netlify)?

Those platforms host static files and short-lived serverless functions (with small size and time limits). TensorFlow is hundreds of megabytes and the model must stay loaded in memory, so it needs a long-running Python server. That is why the architecture is split and connected over HTTPS with CORS.

4 Which dataset did you use?

"Chest X-Ray Images (Pneumonia)" by Kermany et al., 2018 (Mendeley Data, also on Kaggle): 5,856 paediatric chest X-rays from Guangzhou Women and Children's Medical Center, labelled Normal or Pneumonia by physicians. I used 4,484 for training, 748 for validation and the official 624 for testing.

B. Data and preprocessing

5 What preprocessing do you apply?

Where in the code: `backend/utils/preprocessing.py`

Convert to grayscale, resize to 160 x 160 with bilinear interpolation, and scale pixel values from 0-255 to 0-1. The scaling is a `Rescaling(1/255)` layer inside the model, so it is impossible to forget at inference time.

6 Why normalise pixel values?

Large input values produce large activations and gradients, which makes training unstable and slow. Putting inputs in a small, consistent range (0-1) helps gradient descent converge faster.

7 Why grayscale and why 160 x 160?

X-rays have no colour information, so three channels would triple the input with no new information. Pneumonia shows up as fairly large hazy regions, so 160 x 160 keeps enough detail while letting the model train on a laptop CPU (I had no GPU, and a 224 x 224 input with a bigger network was far too slow to train). The Grad-CAM layer is 20 x 20, fine enough to localise regions. A higher resolution is an easy future improvement with a GPU.

8 The original dataset's validation folder has only 16 images. What did you do?

16 images give a very noisy validation score, which makes early stopping unreliable. I created my own validation split: about 15% of the training pool (748 images). The official test set was not touched. (`ml/preprocess.py`)

9 What is a patient-grouped split and why does it matter?

One patient can have several X-rays (the filename contains the patient ID). If one image of a patient is in training and another is in validation, the model can "recognise the patient" and the validation score is inflated - this is data leakage. I used `StratifiedGroupKFold` so that all images of a patient go to the same side while keeping the class ratio the same.

10 What is data augmentation and which ones did you use?

Where in the code: `ml/train.py`

Creating randomly modified copies of training images so the model sees more variety and overfits less. I used small rotations (about 10 degrees), shifts, zoom, brightness and contrast changes. It is applied only to training data.

11 Why no horizontal flip?

Anatomy is not symmetric - the heart is on the left. A flipped X-ray is anatomically wrong, so I only used augmentations that could happen in a real scan (slightly rotated patient, different exposure).

12 The dataset is imbalanced. How did you handle it?

Training has about 3 pneumonia images per normal image. I used class weights ("balanced" from scikit-learn): errors on the rarer Normal class count about 2.9 times more in the loss than errors on Pneumonia. I also report precision, recall and F1 per class instead of only accuracy.

C. The CNN

13 Describe your model architecture.

Where in the code: `ml/train.py`, `build_model`

Input 160x160x1 -> Rescaling -> four convolutional blocks with 16, 32, 64 and 128 filters. Each block is Conv 3x3 -> BatchNorm -> ReLU -> 2x2 max-pooling, so the feature maps go 160 -> 80 -> 40 -> 20 -> 10. Then global average pooling, dropout 0.4, a dense layer with 64 units, dropout 0.3, and one sigmoid output neuron. About 106,000 parameters - deliberately small, because the dataset is small (about 4,500 training images) and it must run on a CPU.

14 What does a convolution layer do?

It slides small filters (3x3) over the image and computes a weighted sum at every position. Each filter learns to detect one pattern (an edge, a texture). The output is a feature map showing where that pattern occurs.

15 Why is a CNN better than a normal (dense) neural network for images?

(1) Local connectivity - a neuron looks at a small patch, matching how image features are local. (2) Weight sharing - the same filter is used everywhere, so far fewer parameters and it detects a pattern wherever it appears. (3) Hierarchy - deeper layers combine simple features into complex ones.

16 What is max-pooling?

It keeps the maximum value in each 2x2 window, halving width and height. It reduces computation, gives a little tolerance to small shifts, and lets deeper layers "see" a larger area of the original image (receptive field).

17 What is batch normalization and why use it?

It normalises the activations of a layer over each mini-batch (zero mean, unit variance) and then learns a scale and shift. It stabilises and speeds up training and has a mild regularising effect. Because BN adds its own shift, I set `use_bias=False` in the conv layers before it.

18 What is dropout?

During training it randomly sets a fraction of the units to zero (40% and 30% here), so the network cannot rely on any single unit. It is switched off at inference. It reduces overfitting.

19 Why global average pooling instead of Flatten?

Flatten on 10x10x128 gives 12,800 values, and a dense layer of 64 units on top would need about 820,000 weights - eight times the rest of the network and easy to overfit. GAP averages each feature map to one number (128 values), which needs far fewer parameters.

20 Why ReLU? Why sigmoid at the output?

ReLU (`max(0, x)`) is cheap and does not saturate for positive values, so gradients flow well in deep networks. The output is sigmoid because this is binary classification: it squashes the output to 0-1, read as P(pneumonia). P(normal) is 1 minus that.

21 Why not softmax with two outputs?

For two classes they are mathematically equivalent. One sigmoid neuron is simpler and has fewer parameters.

22 Which loss and optimizer?

Binary cross-entropy, which penalises confident wrong predictions heavily. Adam optimizer with learning rate 0.001; `ReduceLROnPlateau` halves the learning rate when validation loss stops improving for 3 epochs.

23 What is early stopping?

Training is stopped when the validation loss has not improved for 5 epochs, and the weights from the best epoch are restored. It prevents overfitting and avoids wasting time.

23b Did you train more than once?

Yes. In the first run the validation curve was very unstable (it collapsed to chance level in some epochs). I traced it to batch normalization's moving averages lagging behind (momentum 0.99), changed the momentum to 0.9 and retrained. I made that decision from the validation curve, and I report both runs: the first scored 72.9% on the test set, the final one 78.8%.

24 Why did you build a custom CNN instead of transfer learning (e.g. ResNet)?

The project objective was to design and understand a CNN from scratch, and a small custom network is fast on a CPU and cheap to host. Transfer learning from ImageNet would probably score a few points higher and is my first suggestion for future work.

D. Evaluation

25 How do you know the model is not overfitting?

I compare training and validation curves (Model Insights page). If training accuracy keeps rising while validation loss rises, that is overfitting. I fight it with augmentation, dropout, batch norm, GAP, early stopping and class weights, and the final score is reported on a separate test set.

25b In the first epochs validation accuracy is stuck at 74% while training accuracy rises. Why?

74% is exactly the share of pneumonia images in the validation set - the model was predicting "pneumonia" for everything. The cause is batch normalization: in training it uses the statistics of the current batch, but at validation time it uses moving averages that need a few hundred steps to catch up. After 3-4 epochs they settle and validation accuracy jumps. It also explains why the validation curve is noisier than the training curve.

26 Define accuracy, precision, recall and F1.

- Accuracy = (TP + TN) / all.
- Precision = TP / (TP + FP): of the cases flagged pneumonia, how many really were.
- Recall (sensitivity) = TP / (TP + FN): of the real pneumonia cases, how many were caught.
- F1 = harmonic mean of precision and recall.

27 Which metric matters most here?

Recall for pneumonia, because a false negative (missed pneumonia) is more dangerous than a false positive (which leads to a doctor taking a second look). But precision still matters: too many false alarms make a tool useless.

28 Explain your confusion matrix.

Rows are the true class, columns the predicted class. On the 624 test images: TN = 108 (normal correctly identified), FP = 126 (normal flagged as pneumonia), FN = 6 (pneumonia missed), TP = 384 (pneumonia caught). So the model almost never misses pneumonia (98.5% recall) but raises many false alarms (only 46% of normal X-rays are recognised as normal). Test accuracy is (108 + 384) / 624 = 78.8%.

29 What is ROC-AUC?

The ROC curve plots true-positive rate against false-positive rate for every possible threshold. AUC is the area under it: the probability that a random pneumonia image gets a higher score than a random normal image. 0.5 is random guessing, 1.0 is perfect. It does not depend on the 0.5 threshold.

30 Validation accuracy is 95% but test accuracy is only 79%. Why?

The validation split comes from the same pool as the training data, while the official test set is known to be different (a "distribution shift"): the model's scores for normal test images are shifted upwards, so many cross the 0.5 threshold. The test set is also more balanced (234 normal / 390 pneumonia) than training, so errors on normal images cost more accuracy. The ROC-AUC on the test set is still 0.93, which means the model ranks pneumonia above normal well - the problem is mostly the threshold position, plus a small network trained at low resolution on a CPU. I did not tune the threshold on the test set, because then the test score would no longer be an honest estimate. I report the number as it is.

30b How would you improve the 79%?

(1) Train on a GPU at higher resolution with a deeper network or transfer learning; (2) stronger augmentation and more varied data so the model is less sensitive to the shift; (3) choose the threshold on a validation set that resembles the deployment data; (4) calibrate the probabilities.

31 What does the decision threshold do?

The model outputs a probability; at 0.5 or above I call it Pneumonia. Lowering the threshold catches more pneumonia (higher recall) but creates more false alarms (lower precision), and vice versa.

E. Grad-CAM

32 What is Grad-CAM and why did you add it?

Where in the code: `backend/utils/gradcam.py`

Gradient-weighted Class Activation Mapping. It makes the model less of a black box by highlighting the image regions that influenced the prediction. In healthcare, an explanation helps a user judge whether to trust a result.

33 How does Grad-CAM work, step by step?

1. Run the image through the network and keep the feature maps of the last conv block (20 x 20 x 128).
2. Compute the gradient of the class score with respect to those feature maps.
3. Average each gradient map over space - one importance weight per channel.
4. Take the weighted sum of the feature maps and apply ReLU (keep only positive influence).
5. Normalise to 0-1, upscale to the image size and overlay it as a colour map.

34 Why the last convolutional layer?

It has the most high-level, meaningful features while still keeping spatial layout. After global average pooling the "where" information is gone.

35 You have a single sigmoid output. How do you explain the "Normal" class?

The score for Pneumonia is `p`, and for Normal it is `1 - p`. I differentiate whichever score matches the predicted class.

36 Limitations of Grad-CAM?

The map is coarse (20 x 20 upscaled), shows correlation with the model's output rather than disease location, and can highlight irrelevant areas. It is a sanity check, not a segmentation.

F. Backend, frontend and deployment

37 Why FastAPI?

It is a modern Python framework: fast, typed, validates data with Pydantic, and generates interactive API docs at `/docs` automatically. Python lets me use TensorFlow directly.

38 How does a prediction request flow?

Where in the code: `backend/routes/predict.py`

Browser sends the image as `multipart/form-data` to `POST /api/predict` -> the file is validated (size, real image format, minimum dimensions) -> preprocessed -> the model predicts -> Grad-CAM is computed -> the API returns JSON with the label, probabilities and heatmap images as base64 PNG.

39 Why do you load the model at startup?

Where in the code: `backend/app/state.py`

Loading takes seconds. Doing it once in FastAPI's lifespan handler and keeping it in memory makes every request fast.

40 What is CORS and how did you configure it?

Browsers block JavaScript on one origin (the Vercel site) from calling another origin (the API on Render) unless the API allows it with CORS headers. The allowed origins are a list in `backend/app/config.py` (local development plus my deployed frontend), and more can be added with the `ALLOWED_ORIGINS` environment variable. Any other website's browser requests are refused.

41 How does the frontend know the backend URL?

From the build-time environment variable `VITE_API_URL`, set on the frontend host (Vercel). In development it is empty and the Vite dev server proxies `/api` to the local backend, so no localhost URL exists in the application code.

42 What do `netlify.toml` and `vercel.json` do?

`netlify.toml` tells Netlify the base folder (`frontend`), the build command (`npm run build`) and the publish folder (`dist`). Both files add a rule that sends every path to `/index.html`, so React Router routes like `/analyze` work when the page is refreshed. Without that rule the host looks for a file named `analyze`, does not find one and returns 404 - I hit exactly this on Vercel.

42b Where is the project deployed, and what problems did you face?

Frontend on Vercel, backend on Render's free tier (512 MB RAM), code on GitHub. Four real problems: (1) Hugging Face Spaces, my first choice for the backend, now needs a paid plan for Docker Spaces, so I moved to Render; (2) the backend uses about 420 MB with TensorFlow loaded, close to Render's limit, so I restricted TensorFlow to one thread in the Dockerfile; (3) my Netlify account had no deploy credits left, so the frontend went to Vercel; (4) the first live version showed "Model server unreachable" because the backend's CORS list did not include the Vercel address.

42c Why is the first request sometimes slow?

Render's free instance sleeps after 15 minutes without traffic. The next request has to start the container and load TensorFlow and the model, which takes about a minute. After that a prediction takes a few seconds.

43 How do you validate uploads?

Twice. In the browser: file type and size, for quick feedback. On the server: size limit, the file is actually opened and verified with Pillow (the extension is not trusted), only JPEG/PNG, and minimum dimensions. The server check is the one that matters for security.

44 Are the metrics on the website hard-coded?

No. `ml/evaluate.py` writes `metrics.json`, the API serves it, and the page renders it. If I retrain, the page updates automatically.

45 Do you store the uploaded images?

No. The image is processed in memory and discarded after the response.

G. Limitations, ethics and future work

46 Can this be used in a hospital?

No. It is trained on paediatric X-rays from one hospital, has not been clinically validated or approved by any regulator, only knows two classes, and cannot tell whether an image is even a chest X-ray. The app says this clearly.

47 What are the main limitations?

Single-source paediatric data (may not generalise to adults or other machines); only two classes; confidence is not calibrated; Grad-CAM is coarse; no clinical context; possible dataset shortcuts (e.g. markers on the film).

48 What would you improve next?

Transfer learning (DenseNet/EfficientNet); training on larger multi-hospital datasets (NIH ChestX-ray14, CheXpert); bacterial vs viral classification; probability calibration (temperature scaling); an "is this a chest X-ray?" check; k-fold cross-validation with confidence intervals; external validation.

49 What ethical issues does medical AI raise?

Patient privacy, bias (a model trained on one population may fail on another), over-reliance by users, accountability for errors, and the need for transparency. That is why the app shows explanations, limitations and a disclaimer, and stores nothing.

50 What was the hardest part?

A good honest answer: getting a trustworthy evaluation. The 16-image validation folder, patient leakage and class imbalance can all make results look better than they are; I had to design the split and metrics carefully.

Quick formula sheet

Term	Formula or meaning
Accuracy	(TP + TN) / (TP + TN + FP + FN)
Precision	TP / (TP + FP)
Recall / sensitivity	TP / (TP + FN)
Specificity	TN / (TN + FP)
F1-score	2 x precision x recall / (precision + recall)
Sigmoid	1 / (1 + e^(-x)), maps any number to the range 0 to 1
ReLU	max(0, x)
Binary cross-entropy	-[y log(p) + (1 - y) log(1 - p)]
Conv output size (same padding)	Same width and height as the input; channels = number of filters
Conv parameters (no bias)	kernel height x kernel width x input channels x filters
Max-pool 2 x 2	Halves width and height
Grad-CAM weight	Average over space of d(score) / d(feature map)
Grad-CAM map	ReLU(sum over channels of weight x feature map)

Worked example with this project's numbers: accuracy = (384 + 108) / 624 = 78.8%; pneumonia recall = 384 / (384 + 6) = 98.5%; specificity = 108 / (108 + 126) = 46.2%; pneumonia precision = 384 / (384 + 126) = 75.3%. First convolution: 3 x 3 x 1 x 16 = 144 parameters.